

Constrained Pseudorandom Functions

Revisited

Soutenance de projet — Master 1 Cryptologie

Julian Mouthon & Mario Razafinony

Sorbonne Université — 24 mai 2026

Plan de la presentation

- 1 Motivations et definitions
- 2 Construction BMO17
- 3 L'attaque CJ25
- 4 Alternatives a RSA
- 5 CPRF hachee et securite
- 6 Resultats experimentaux
- 7 Conclusion

Pourquoi les CPRF ?

Probleme concret

- On veut deleguer l'évaluation d'une PRF sur un sous-ensemble d'entrees
- Sans reveler la cle secrete complete
- Applications : *Searchable Symmetric Encryption*, stockage externalise

Solution : les CPRF

- Une **PRF** : sortie indiscernable du hasard
- Une **CPRF** : restreint l'évaluation par une contrainte C
- La cle contrainte K_C ne permet pas de retrouver K

Analogie

Comme un **badge d'accès partiel** : il ouvre certaines portes, pas toutes, sans reveler le passe maitre.

Notre contrainte principale

$$C_n(x) = [x < n]$$

Autoriser les entrees $< n$.

Definition formelle d'une CPRF

$\text{Setup}(1^\lambda) \rightarrow K$

Genere la **cle maitre**
 K

$\text{Constrain}(K, C) \rightarrow K_C$

Cree une **cle**
contrainte pour C

$\text{Eval}(K, x) \rightarrow y$

Evalue avec la **cle**
maitre

$\text{EvalC}(K_C, x) \rightarrow y$

Evalue avec K_C si
 $C(x) = 1$

Propriete de correction

$\forall x$ tel que $C(x) = 1$:

$$\text{EvalC}(K_C, x) = \text{Eval}(K, x)$$

La cle contrainte donne le meme resultat que
la cle maitre sur les entrees autorisees.

Securite intuitive

Connaitre K_C ne doit **pas** permettre de
calculer $\text{Eval}(K, x)$ pour x tel que $C(x) = 0$.

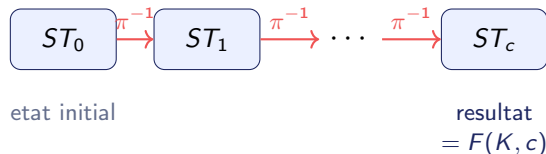
Autrement dit : la cle contrainte ne revele
rien sur les entrees interdites.

Idee centrale

- Permutation a trappe π basee sur RSA
- Etat initial secret ST_0
- Evaluer en c = appliquer π^{-1} exactement c fois

Cle maitre : (ST_0, SK)

$$\text{Eval}((SK, ST_0), c) = \pi_{SK}^{-c}(ST_0)$$



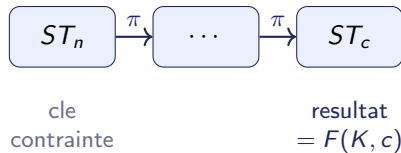
Cle contrainte pour $C(x) = [x < n]$

- Cle publique PK
- Etat $ST_n = \pi_{SK}^{-n}(ST_0)$
- Seuil n

⇒ La cle secrete SK n'est **jamais** incluse

Evaluation contrainte ($c < n$) :

$$\text{EvalC}((PK, ST_n, n), c) = \pi_{PK}^{n-c}(ST_n)$$



Correction

$$\pi^{n-c}(ST_n) = \pi^{n-c}(\pi^{-n}(ST_0)) = \pi^{-c}(ST_0)$$

✓

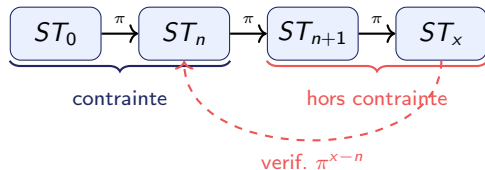
Idee de l'attaque

- 1 Demander la cle contrainte (PK, ST_n, n)
- 2 Interroger l'oracle sur $x > n$, obtenir ST_x
- 3 Calculer $\pi_{PK}^{x-n}(ST_x)$ et comparer a ST_n
- 4 Si egal $\Rightarrow$ c'est une CPRF !

Avantage de l'attaquant

$$\Pr[\mathcal{A} \text{ distingue}] \approx 1 - \frac{1}{N}$$

Negligeable seulement si N est exponentiel.



Architecture client–serveur TCP

Serveur = Oracle

- $\text{EVAL}(x)$: retourne $F(K, x)$ ou valeur aleatoire selon le monde
- $\text{CONSTRAIN}(n)$: retourne (PK, ST_n, n)

Client = Attaquant

- Choisit n , demande la cle contrainte
- Fait des requetes $x > n$
- Verifie la coherence via π^{x-n}

Resultat

L'attaque reussit avec probabilite ≈ 1 des la premiere requete sur la version **non** hachee.

Conclusion

BMO17 sans hachage **n'est pas** IND-securisee. La structure algebrique est completement exploitable.

Alternatives a la permutation RSA

F^{WEAK} — lineaire

$$\text{Eval}(S, \vec{x}) = S \cdot \vec{x}$$

$$S \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$$

Rabin

$$f(x) = x^2 \bmod n$$

Basee sur la factorisation.

AES

$$f(x) = E_k(x)$$

Bijection sur blocs 128 bits.

Cassee

Systeme lineaire soluble avec $N - 1$ requetes : on retrouve S entierement. **Pas de trappe iterative.**

Non injective

4 pre-images par image : impossible d'iterer f^{-1} de facon deterministe.
Ambiguite a chaque inversion.

Sans trappe publique

Inverser necessite k : pas de separation public/privé. **Pas de cle contrainte possible.**

Bijectivite + Trappe public/privé + Efficacite $\Rightarrow$ Seul RSA satisfait les trois.

Attaque par systeme lineaire

- Choisir $\vec{y} = (0, \dots, 0, 1)$, obtenir $S_{\vec{y}}$
 - Calculer $k_i = S_{\vec{y}_i} \cdot \vec{x}_j$ pour $N - 1$ vecteurs $\vec{x}_j \perp \vec{y}$
 - Resoudre $X \cdot \vec{S}_i^T = \vec{k}_i$: systeme $(N - 1) \times (N - 1)$ inversible
- $\Rightarrow$ On retrouve S en entier avec 1 requete d'evaluation

F^{WEAK} est cassee

Un seul appel a EVAL suffit a retrouver la cle maitre.

Renforcement par hachage

$$\text{Eval}_H(\vec{x}) = H(S \cdot \vec{x})$$

Retrouver S necessite une pre-image de H
 $\Rightarrow$ infaisable.

Mais une limite demeure

F^{WEAK} n'est **pas** IND*-NC-CPRF securisee. Le theoreme de CJ25 ne peut donc pas s'appliquer directement pour prouver la securite de $\text{Hashed}[F^{\text{WEAK}}]$.

Renforcement par hachage — Principe

Idee : appliquer un oracle aleatoire P sur la sortie de la CPRF

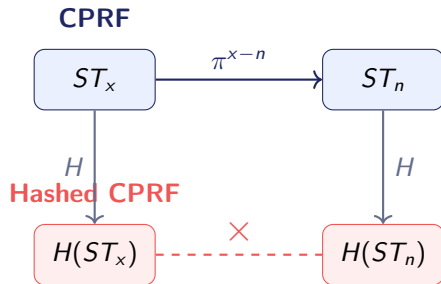
$$\text{Eval}_H(K, x) = P(\text{Eval}(K, x))$$

Lazy Sampling : P attribue une valeur aleatoire a chaque entree, memorisee

- Si $z \notin \sigma_P$: choisir $y \leftarrow \{0, 1\}^\lambda$, memoriser (z, y)
- Sinon : retourner le y memorise

Deux implementations

- Version SHA-256 (pratique)
- Version lazy sampling (modele theorique CJ25)



Effet cle

L'attaquant ne peut plus verifier $\pi^{x-n}(ST_x) = ST_n$ car il ne voit que les haches.

Theoreme (Cheng–Jaeger 2025)

Si les trois conditions suivantes sont satisfaites :

- ① $\min_x |\text{CF.Out}(\lambda, x)|$ est super-polynomial en λ
- ② CF est $\text{IND}^*\text{-NC-CPRF}$ securisee
- ③ L'oracle aleatoire est **non-programmable**

Alors $\text{Hashed}[\text{CF}]$ est $\text{SIM}^*\text{-AC-CPRF}$ securisee.

Application a BMO17 (RSA)

- ✓ Espace de sortie $\approx 2^{4096}$: super-polynomial
- ✓ RSA est $\text{IND}^*\text{-NC}$ securisee
- ✓ Oracle non-programmable (lazy sampling)

Bilan

$\text{Hashed}[\text{BMO17}]$ est **prouvablement securisee** contre l'attaque CJ25.

Application a F^{WEAK}

× Condition (2) non satisfaite
⇒ Le theoreme ne s'applique pas.

Configuration materielle

Composant	Detail
Machine	Dell Inspiron 3501
CPU	Intel Core i7-1165G7 4 coeurs, 8 threads
Frequence	2.80 GHz (jusqu'a 4.70)
RAM	7.5 GB
OS	Ubuntu 24.04.4 LTS
Noyau	Linux 6.8.0
Architecture	x86_64
GPU	Aucun

Protocole experimental

- 100 requetes d'evaluation par variante
- Taux de succes de l'attaque CJ25 mesure en fonction du nombre de requetes
- 4 variantes testees :
 - CPRF BMO17 non hachee
 - CPRF BMO17 hachee SHA-256
 - CPRF BMO17 hachee lazy sampling
 - F^{WEAK} non hachee

Resultats experimentaux — Observations

CPRF non hachee

Taux ≈ 1 des la **1ère requete**.
Structure algebrique directement exploitable.

CPRF hachee SHA-256

Taux ≈ 0.5 : decision **aleatoire**.
L'attaquant ne distingue plus rien.

F^{WEAK} non hachee

Avantage significatif visible rapidement.
Matrice S retrouvee en quelques requetes.

CPRF hachee lazy sampling

Taux ≈ 0.5 egalement.
Resultats identiques a SHA-256.

Conclusion

Le hachage (SHA-256 ou lazy sampling) **neutralise complètement** l'attaque CJ25.
Les deux approches sont **equivalentes** en pratique.

Conclusion

Ce que nous avons realise

- BMO17 avec RSA 4096 bits (OpenSSL)
- Attaque CJ25 en client–serveur TCP
- Analyse : F^{WEAK} , Rabin, AES
- CPRF hachee : SHA-256 et lazy sampling
- Etude de Hashed[F^{WEAK}]

Resultats principaux

- Le hachage **neutralise** l'attaque CJ25
- RSA : seule permutation a trappe adaptee
- F^{WEAK} cassee mais instructive

Perspectives

- Contraintes plus expressives (intervalles, circuits)
- Constructions : LWE, couplages
- Preuve formelle de Hashed[F^{WEAK}]

Code source

github.com/julian-mtn/bmo17-cprf

Merci pour votre attention ! Questions ?